

Background

Zoomerbinis are mystical creatures known for their brilliant minds, particularly in solving logic puzzles. Like their [forebearers](#), each one has a distinct combination of attributes that defines their style and unique character. Unfortunately, while brimming with potential, they're a bit of a shy bunch. They're struggling to form optimal study groups for their university subjects, and require a (would you guess it) *Python programmer* to come to their rescue!

Thankfully, the Zoomerbinis have your help and guidance. Your challenge in this project is to write some Python code to help them compute an optimal set of study groups, thus enabling them to reach their full potential!



Three Zoomerbinis

Zoomerbinis

There are 256 "types" of Zoomerbinis, each having a corresponding numeric identifier (a "type ID") represented as an integer between 0 and 255 inclusive. The figure below depicts the full mapping. You can also view an enlarged version of the same figure [here](#).

Zoomerbini Type

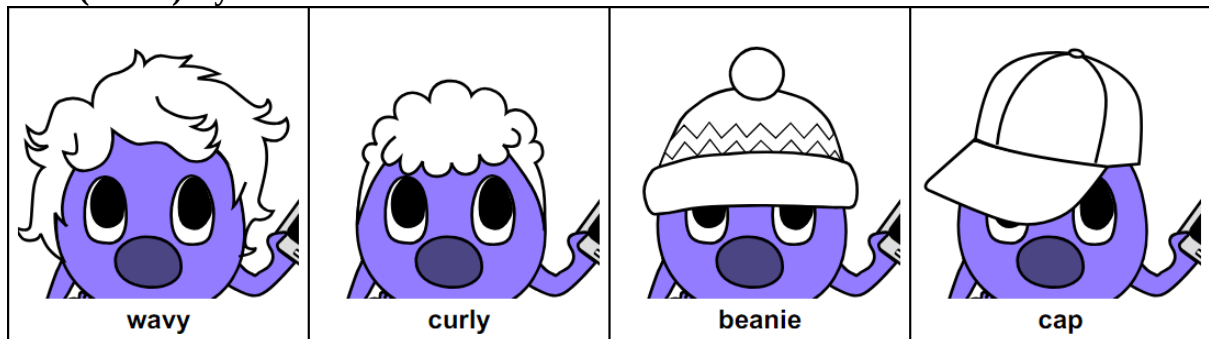
ID Mapping (click [here](#) for enlarged version)

If you study the mapping closely, you'll notice that it is far from random – there are certain attributes that different subsets of Zoomerbini types share, and these can be inferred from the type ID numbers themselves. Indeed, it is the unique *combination* of features that defines any one type.

Specifically, there are **four** attributes, each of which can take exactly one of four values to uniquely define a Zoomerbini type. First, each one will have a unique hair style or choice of hat. Second, each has a favourite colour (note that a Zoomerbini will always either dye their hair that colour or wear a hat of that same colour!). Third, each has a preferred social media platform. Finally, each has a "fashion accessory" for some extra flair!

Here are the possible values and corresponding visualisations for each attribute:

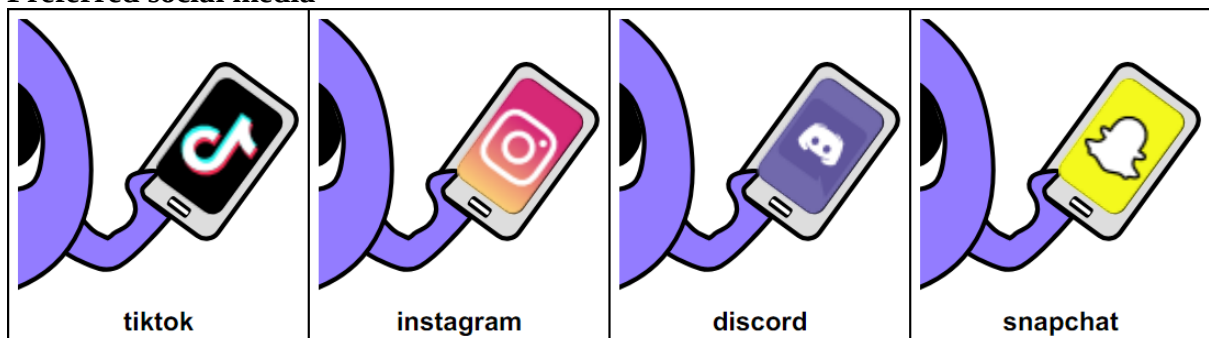
Hair (or hat) style



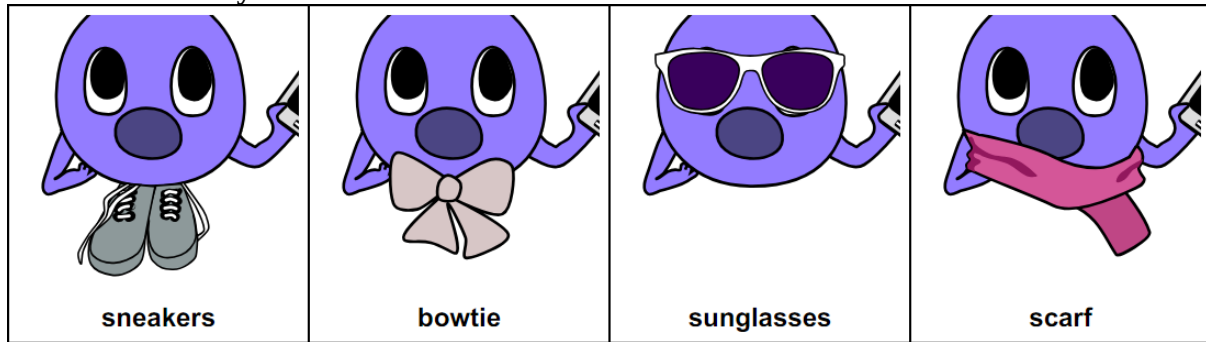
Favourite colour



Preferred social media



Fashion accessory



Task 1: Decoding Attributes (3 marks)

Write a function `zbini_attrs(type_id)` that converts a Zoomerbini type ID to a tuple of strings representing the four underlying attributes in the following order: Hair/hat style, favourite colour, fashion accessory and preferred social media platform.

Here are the possible values (as string literals) for each attribute:

- Hair/Hat: "wavy", "curly", "cap" or "beanie"
- Colour: "red", "blue", "yellow" or "green"
- Accessory: "sneakers", "bowtie", "scarf" or "sunglasses"
- Social: "tiktok", "instagram", "snapchat" or "discord"

The parameter `type_id` may be assumed to be a numeric whole number value (integer). The returned attributes for a given ID should match those shown in the full [mapping table](#) in all cases.

If a `type_id` outside the valid range is given, the function should instead return `None`.

Hint: There are multiple ways to solve this problem, but regardless of which one you pick, ensure that it is not overly redundant. In particular, you should be trying to identify pattern(s) in the way that type ID numbers map to Zoomerbini attributes and computing a result based on these in a concise and well structured manner.

Example Calls:

```
>>> print(zbini_attrs(0))
('wavy', 'red', 'sneakers', 'tiktok')
>>> print(zbini_attrs(96))
('curly', 'yellow', 'sneakers', 'tiktok')
>>> print(zbini_attrs(351))
None
```

Task 2: Checking Group Validity (3 marks)

While Zoomerbins are usually quite amicable with one another, they have a rather specific set of preferences when it comes to forming study groups! Specifically, a study group is **valid** if for **each** of the four attributes:

- All members have the same attribute value, *or*;
- All members have unique attribute values, i.e. no two members share the same attribute value.

Additionally,

- A valid group can only have either 3 or 4 members, *and*;
- Each member must study at least one subject in common with every other member of the group.

Write a function `valid_study_group(zbinis, group)` that computes the validity of a given study group of Zoomerbini, as per the above definition.

The parameter `zbinis` will be a list of Zoomerbini, each represented as a `(type_id, subjects)` tuple, where `type_id` is a Zoomerbini type ID (0 to 255 inclusive) and `subjects` is a list of non-empty strings denoting the subjects the respective Zoomerbini is studying.

The parameter `group` will be a tuple of *indices* into the `zbinis` list and reflects the group of Zoomerbini being checked for validity (`zbinis` may be of a different length to `group`). Note that while Zoomerbini in a group may share the same type IDs, they must be *distinct* individuals in `zbinis`. In other words, if an index appears more than once in `group`, the group is invalid.

Your function should return a tuple containing two elements:

- A boolean value denoting the *validity* of the respective group (`True` if the group is valid or `False` if not).
- A positive integer representing the number of subjects all members of the group share in common with one another, or otherwise `None` if the group is not valid.

A working version of the `zbin_attrs(type_id)` function from Task 1 has been provided to help you with this task.

Example Calls:

```
>>> example_zbinis = [(0, ['FoC', 'Calc 1', 'Logic']), (108, ['FoA', 'Calc 2', 'Logic']), (148, ['FoC', 'Calc 1', 'Logic']), (248, ['FoC', 'Calc 1', 'Logic']), (0, ['Calc 2', 'History', 'Politics']), (108, ['FoC', 'Calc 1', 'Logic'])]
>>> valid_study_group(example_zbinis, (0, 1, 3))
(True, 1)
>>> valid_study_group(example_zbinis, (0, 1, 5))
(False, None)
>>> valid_study_group(example_zbinis, (0, 1, 2, 3))
(True, 1)
>>> valid_study_group(example_zbinis, (1, 2, 3, 4))
(False, None)
```

Task 3: Possible Study Groups (4 marks)

Write a function `possible_study_groups(zbinis)`. The parameter `zbinis` is a list of Zoomerbinis of the same form as in Task 2.

The function should return a list representing all the possible ways a **valid** study group could be chosen from `zbinis`. Each element is a tuple where:

- The first element is a tuple of indices representing a group. This tuple should contain exactly three or four unique indices into the `zbinis` list in ascending order.
- The second element is a **score** for the group, derived using a "points" system (described below).

For example, if `zbinis` was of length four, there could be *up to* five tuples of indices in the result with the first element in each being: `(0, 1, 2, 3)`, `(0, 1, 2)`, `(0, 1, 3)`, `(0, 2, 3)` and `(1, 2, 3)`

Each group's score (the second element in each returned tuple) is derived using a "points" system according to the following rules:

- Add **three** points for each subject shared by all members of the group, *and*;
- Add **one** point if the group has three members, or **two** points if the group has four members.

The returned groups should be sorted in **descending** order using the above scoring system, that is, higher scored groups should come before lower scored groups in the resulting list.

If there is ever a tie in scores, the group's indices should be used as a tiebreaker. Each index should be considered in turn (left to right) until one facilitates a tiebreak. For example, the grouping `(0, 2, 3)` should come before `(1, 2, 3)`, and `(0, 1, 2)` should come before `(0, 1, 3)`.

Groups that are invalid as per the same rules as in Task 2 should be excluded from the returned list altogether. A working version of the respective `valid_study_group(zbinis, group)` function has been provided to help you with this task.

Example Calls:

```
>>> print(possible_study_groups([(198, ['FoC']), (138, ['FoC', 'Calc 1']),
(14, ['FoC', 'Calc 1']), (66, ['Calc 1'])]))
[(0, 1, 2), 4], ((1, 2, 3), 4)]
>>> print(possible_study_groups([(198, ['FoC']), (138, ['FoC', 'Calc 1']),
(14, ['FoC']), (66, ['FoC'])]))
[((0, 1, 2, 3), 5), ((0, 1, 2), 4), ((0, 1, 3), 4), ((0, 2, 3), 4), ((1, 2,
3), 4)]
>>> print(possible_study_groups([(198, ['FoC']), (138, ['Calc 1']), (14,
['Calc 1']), (66, ['FoC'])]))
[]
```

Task 4: Allocating Study Groups (5 marks)

In this task you will finally allocate the Zoomerbinis to their study groups!

Write a function `alloc_study_groups(zbinis)` where the parameter `zbinis` is a list of Zoomerbinis, each represented as a `(type_id, subjects)` tuple (this is again of the same form as in the previous two tasks). The function should compute a set of groups of `zbinis` such that the number of Zoomerbinis without a group is **minimised**. We call this an *optimal* grouping.

The return value must be a list of tuples of indices into the `zbinis` list, where each tuple corresponds to a valid group in descending score order (following the same ordering rules as in Task 3). Importantly, a Zoomerbini cannot be allocated to more than one group, that is, any index in the result should appear at most once.

In the case that there are multiple optimal groupings, the grouping with the highest *combined score* out of these should be returned. This score is computed by summing up all the individual group scores that make up a given grouping (each group should be scored using the same points system as described in Task 3).

If there is still a tie between optimal groupings after considering combined scores, your function should preference the grouping with the smallest *minimum index*. For example, in the grouping `[(2, 3, 4), (6, 7, 9)]` the minimum index is 2, and hence should be preferred over `[(3, 4, 5), (6, 7, 9)]` which has a minimum index of 3. If more than one optimal grouping has the same minimum index, the second-to-minimum index should be used to tiebreak instead (or the third-to-minimum if there's a tie on both the minimum and second-to-minimum indices, etc). No further tiebreaking beyond this point is required.

Since this is a computationally expensive problem to solve, you may assume the length of `zbinis` will be at most 10. A working version of the `possible_study_groups(zbinis)` function has been provided to help you with this task.

Example Calls:

```
>>> print(alloc_study_groups([(198, ['FoC']), (198, ['FoC']), (138,
['FoC']), (14, ['FoC'])]))
[(0, 2, 3)]
>>> print(alloc_study_groups([(198, ['FoC']), (138, ['FoC', 'Calc 1']),
(14, ['FoC', 'Calc 1']), (66, ['FoC', 'Calc 1'])]))
[(0, 1, 2, 3)]
>>> print(alloc_study_groups([(198, ['FoC', 'Logic']), (138, ['Calc 1']),
(14, ['Calc 1']), (66, ['FoC', 'Logic']), (10, ['FoC', 'Logic']), (142,
['FoC', 'Logic']), (66, ['Calc 1'])]))
[(0, 3, 4, 5), (1, 2, 6)]
```